JPWP • PROJEKT

Współbieżność i wielowątkowość w Pythonie

Mateusz Maślanka • Arkadiusz Baran
AGH Akademia Górniczo-Hutnicza • Teleinformatyka

Agenda

01 **Współbieżność vs równoległość** czym różnią się te pojęcia

02 **Wątki, procesy i GIL** ograniczenia interpretera CPython

03 **Tworzenie wątków i ich cykl życia** moduł threading, stany wątku

04 **Race condition i synchronizacja** problem wyścigu, Lock, sekcja krytyczna

05 **Komunikacja przez Queue** producer-consumer, graceful shutdown

06 **Deadlock i antypatterns** pułapki wielowątkowości

07 **ThreadPoolExecutor** pula wątków z concurrent.futures

08 **Case study i benchmarki** zastosowania w telekomunikacji

Współbieżność vs równoległość

WSPÓŁBIEŻNOŚĆ

Wiele zadań wykonywanych naprzemiennie

Jedno CPU przełącza kontekst między zadaniami - powstaje iluzja jednoczesności.

Cel:

efektywne wykorzystanie czasu oczekiwania (I/O, sieć, dysk).

Przykład:

serwer WWW obsługujący wielu klientów jednocześnie.

RÓWNOLEGŁOŚĆ

Zadania wykonywane jednocześnie

Wiele rdzeni CPU pracuje naprawdę równolegle - wymaga sprzętu wielordzeniowego.

Cel:

przyspieszenie obliczeń intensywnych (CPU-bound).

Przykład:

renderowanie klatek animacji na wielu rdzeniach.

Wątki, procesy i GIL

WĄTEK
Thread

- Współdzieli pamięć procesu
- Niski koszt utworzenia
- Idealny dla zadań I/O-bound

PROCES
Process

- Własna przestrzeń adresowa
- Wyższy koszt utworzenia
- Idealny dla zadań CPU-bound

GIL
Global Interpreter Lock

- Tylko 1 wątek naraz wykonuje bytecode
- Zwalniany podczas operacji I/O
- Dlatego wątki pomagają w I/O

W skrócie: GIL blokuje równoległość wątków w Pythonie - dla CPU-bound używaj procesów, dla I/O-bound wątki wystarczą.

Tworzenie wątków - threading.Thread

```python
import threading

def zadanie(numer):
    print(f"Wątek {numer} działa")

# utworzenie i start 5 wątków
watki = [
    threading.Thread(target=zadanie, args=(i,))
    for i in range(5)
]

for t in watki: t.start()
for t in watki: t.join()
```

Kluczowe metody

`Thread(target, args)`

konstruktor wątku

`start()`

uruchamia wątek

`join()`

czeka na zakończenie

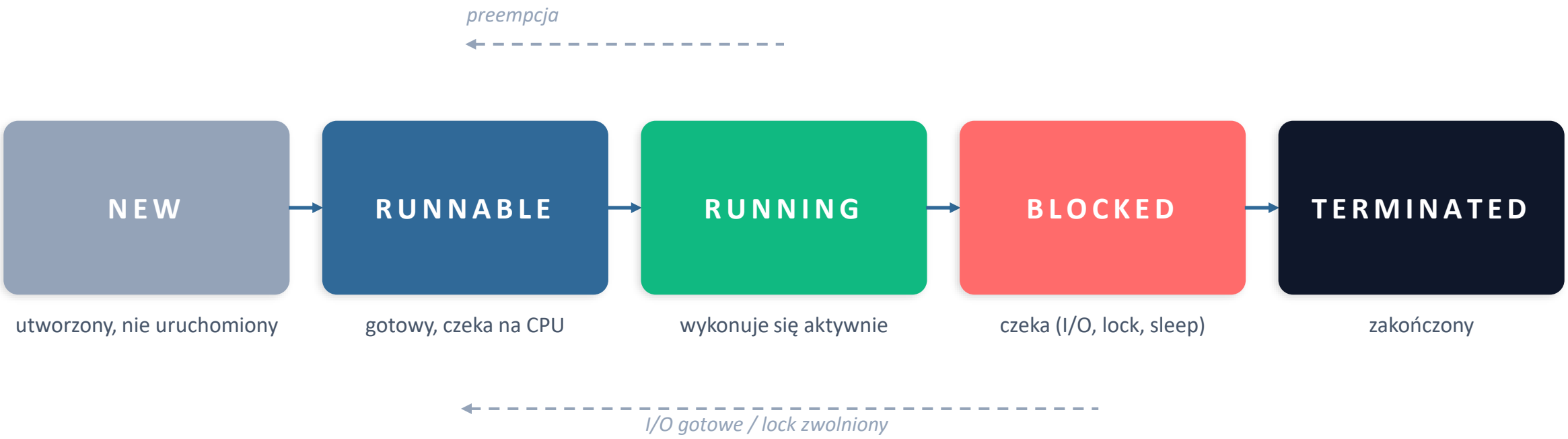
`is_alive()`

czy wątek wciąż żyje

`daemon=True`

kończy się z programem

Cykl życia wątku



Pamiętaj: stan BLOCKED to wątek czekający na warunek - scheduler go obudzi gdy warunek zostanie spełniony.

Race condition - problem wyścigu

```python
import threading
import time

counter = 0

def increment():
    global counter
    for _ in range(1000):
        current = counter
        time.sleep(0.0001)   # symulacja opóźnienia
        counter = current + 1

# 5 wątków zwiększa ten sam licznik
watki = [threading.Thread(target=increment) for _ in
range(5)]
for t in watki: t.start()
for t in watki: t.join()

print(f"Wynik: {counter}  (oczekiwano: 5000)")
```

```
$ python counter.py
Wynik: 4731    Oczekiwano: 5000  X
```

Dlaczego?

Operacja **`counter += 1`** NIE jest atomowa.

Wątki czytają tę samą wartość, inkrementują ją i nadpisują się nawzajem - aktualizacje giną.

Potrzebujemy sekcji krytycznej.

Naprawa race condition - z Lockiem

```python
import threading

counter = 0
lock = threading.Lock()

def increment():
    global counter
    for _ in range(1000):
        with lock:
            current = counter
            counter = current + 1

watki = [threading.Thread(target=increment) for _ in
range(5)]
for t in watki: t.start()
for t in watki: t.join()

print(f"Wynik: {counter}  (oczekiwano: 5000)")

# Wynik: 5000  (oczekiwano: 5000)
```

Co się zmieniło?

Wynik: 5000 (zawsze)

żadnych zgubionych aktualizacji

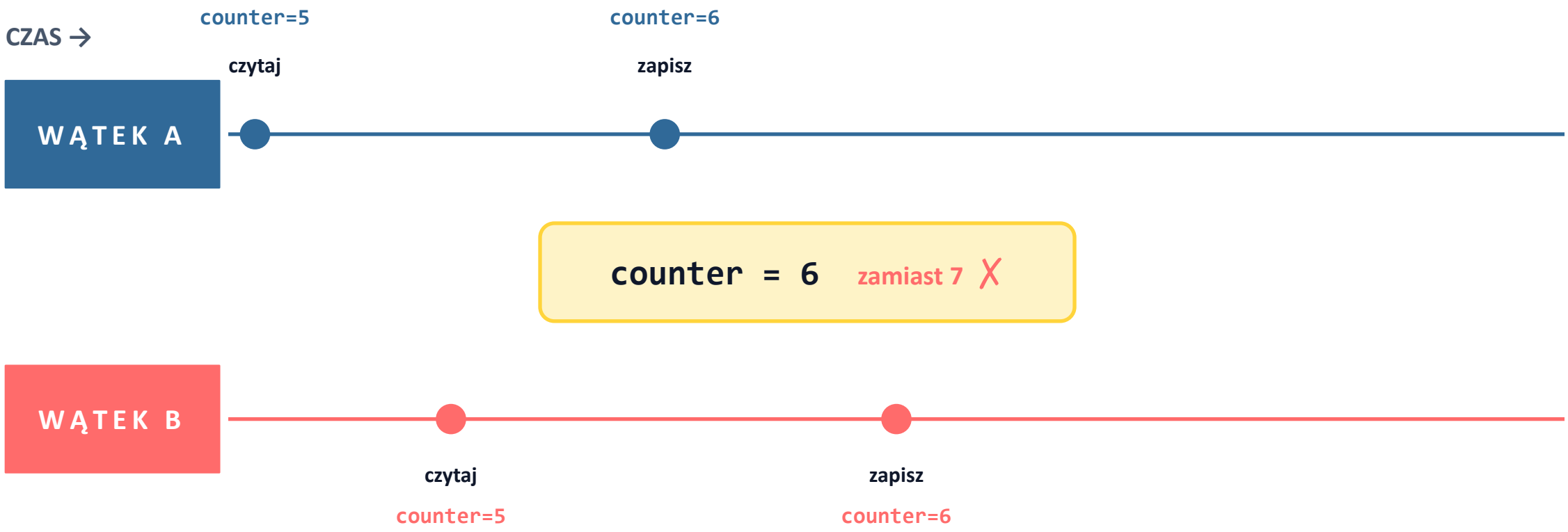
Klucz: with lock

tylko 1 wątek naraz w sekcji krytycznej

Minimalna sekcja krytyczna

tylko odczyt + zapis counter w lockowanym bloku

Wizualizacja race condition



Oba wątki przeczytały **`counter=5`** i niezależnie zapisały **6**. Jedna inkrementacja znikła - **powinno być 7, jest 6.**

threading.Lock - sekcja krytyczna

```python
import threading

lock = threading.Lock()

def bezpieczna_operacja():
    with lock:
        # sekcja krytyczna - tylko 1 wątek naraz
        ...

# alternatywnie (rzadko używane, gorsze przy
wyjątkach):
lock.acquire()
try:
    ...
finally:
    lock.release()
```

Jak używać?

1. Utwórz blokadę

```
lock = threading.Lock()
```

2. Zabezpiecz sekcję

```
with lock:  # auto-release
    ...
```

3. Minimalizuj sekcję

im krócej trzymany lock, tym lepsza wydajność

Chroń powiązane zmienne razem

```python
import threading

saldo = 1000
historia = []
lock = threading.Lock()

def wplata(kwota):
    global saldo
    with lock:
        # oba zapisy to JEDNA sekcja krytyczna
        saldo += kwota
        historia.append(f"+{kwota}")
```

Pułapka

Jeśli dwie operacje muszą pozostać spójne (np. dopisanie do listy + inkrementacja licznika) - oba kroki należą do **tej samej sekcji krytycznej.**

Dwa osobne locki = inny wątek może się wklinować między operacjami i **zepsuć spójność stanu.**

queue.Queue - komunikacja między wątkami

```python
import queue
import threading

q = queue.Queue()

def producer():
    for item in ["a", "b", "c"]:
        q.put(item)

def consumer():
    while True:
        item = q.get()
        print(f"Przetwarzam: {item}")
        q.task_done()

threading.Thread(target=producer).start()
threading.Thread(target=consumer,
daemon=True).start()
q.join()   # czeka aż wszystkie zadania przetworzone
```

API Queue

`put(x)`

dodaje element (blokuje gdy pełna)

`get()`

pobiera element (blokuje gdy pusta)

`task_done()`

sygnalizuje koniec zadania

`join()`

czeka aż kolejka opustoszeje

Graceful shutdown workerów

Problem

Worker w pętli `while True:` czeka na nowe zadania.

Gdy zadania się skończą, wywołanie `queue.get()` będzie **blokować w nieskończoność** - program nie wróci do terminala.

Rozwiązanie

Wprowadź **sygnał końca** (np. **None**).

Worker odczytując sygnał - wychodzi z pętli.

Wskazówka:

wstaw do kolejki **tyle sygnałów końca ilu jest workerów** - każdy odbierze swój i zakończy pracę.

Graceful shutdown - kod z sygnałem None

```python
import queue, threading

q = queue.Queue()
LICZBA_WORKEROW = 3

def consumer():
    while True:
        item = q.get()
        if item is None:        # sygnał końca
            q.task_done()
            break
        print(f"Przetwarzam: {item}")
        q.task_done()

# Start workerów
for _ in range(LICZBA_WORKEROW):
    threading.Thread(target=consumer).start()

# Dodaj zadania
for item in ["a", "b", "c", "d"]:
    q.put(item)

# Sygnały końca - po jednym dla każdego workera
for _ in range(LICZBA_WORKEROW):
    q.put(None)

q.join()  # czekamy aż wszystkie zadania przetworzone
```

Sygnał końca

`Producer wstawia None`

po skończonej pracy

`Consumer wykrywa None`

i wychodzi z pętli

`Tyle None ile workerów`

każdy odbierze swój sygnał

`Brak daemon=True`

workery skończą czysto, bez zabicia

`q.join() w main`

czeka aż kolejka opustoszeje

Deadlock - zakleszczenie

```python
import threading
import time

lock_a = threading.Lock()
lock_b = threading.Lock()

def watek_1():
    with lock_a:
        time.sleep(0.1)
        with lock_b:        # czeka na B
            print("Wątek 1")

def watek_2():
    with lock_b:
        time.sleep(0.1)
        with lock_a:        # czeka na A
            print("Wątek 2")
```

Jak unikać deadlocka?

1. **Stała kolejność blokad**
   wszyscy zdobywają locki w tej samej kolejności
2. **Timeout przy acquire()**
   acquire(timeout=2) - nie blokuj wiecznie
3. **Jak najmniej blokad**
   im mniej locków trzymanych naraz, tym bezpieczniej
4. **Rozważ inne prymitywy**
   Queue często eliminuje potrzebę blokad

Deadlock - jak go naprawić w kodzie

```python
import threading

lock_a = threading.Lock()
lock_b = threading.Lock()

# REGUŁA: ZAWSZE najpierw lock_a, potem lock_b
def watek_1():
    with lock_a:
        with lock_b:
            print("Wątek 1 - bezpiecznie")

def watek_2():
    with lock_a:        # ← TEŻ najpierw A!
        with lock_b:    # ← potem B
            print("Wątek 2 - bezpiecznie")

# Cykl niemożliwy → brak deadlocka
```

Recepta

1. Ustal globalny porządek

`np. zawsze: lock_a → lock_b`

2. Stosuj go w KAŻDYM wątku

`żaden wątek nie może wziąć b przed a`

`...`

3. Cykl staje się niemożliwy

nie ma jak utworzyć łańcucha czekania

ThreadPoolExecutor - pula wątków

```python
from concurrent.futures import ThreadPoolExecutor
import time

def zadanie(x):
    time.sleep(0.5)
    return x * x

with ThreadPoolExecutor(max_workers=4) as executor:
    wyniki = list(executor.map(zadanie, range(10)))

print(wyniki)  # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Zalety puli

- **Bez ręcznego zarządzania**

  with automatycznie uruchamia i kończy wątki

- **Ograniczenie równoległości**

  max_workers kontroluje zużycie zasobów

- **Obiekt Future**

  submit() zwraca Future - result(), done()

- **Metoda map()**

  wygodna dla listy zadań tego samego typu

Future i submit() - obietnica wyniku

```python
from concurrent.futures import ThreadPoolExecutor
import time

def pobierz(url):
    time.sleep(1)
    return f"dane z {url}"

with ThreadPoolExecutor(max_workers=3) as executor:
    # submit() zwraca Future - zadanie startuje od razu
    future_a = executor.submit(pobierz, "a.com")
    future_b = executor.submit(pobierz, "b.com")

    print("Robię coś innego...")  # nie czekamy na wynik

    # result() blokuje aż wynik dostępny
    print(future_a.result())  # "dane z a.com"
    print(future_b.result())  # "dane z b.com"
```

API Future

`submit(fn, *args)`

wysyła zadanie, zwraca Future

`future.result()`

pobiera wynik (czeka jeśli trzeba)

`future.done()`

sprawdza czy zadanie skończone

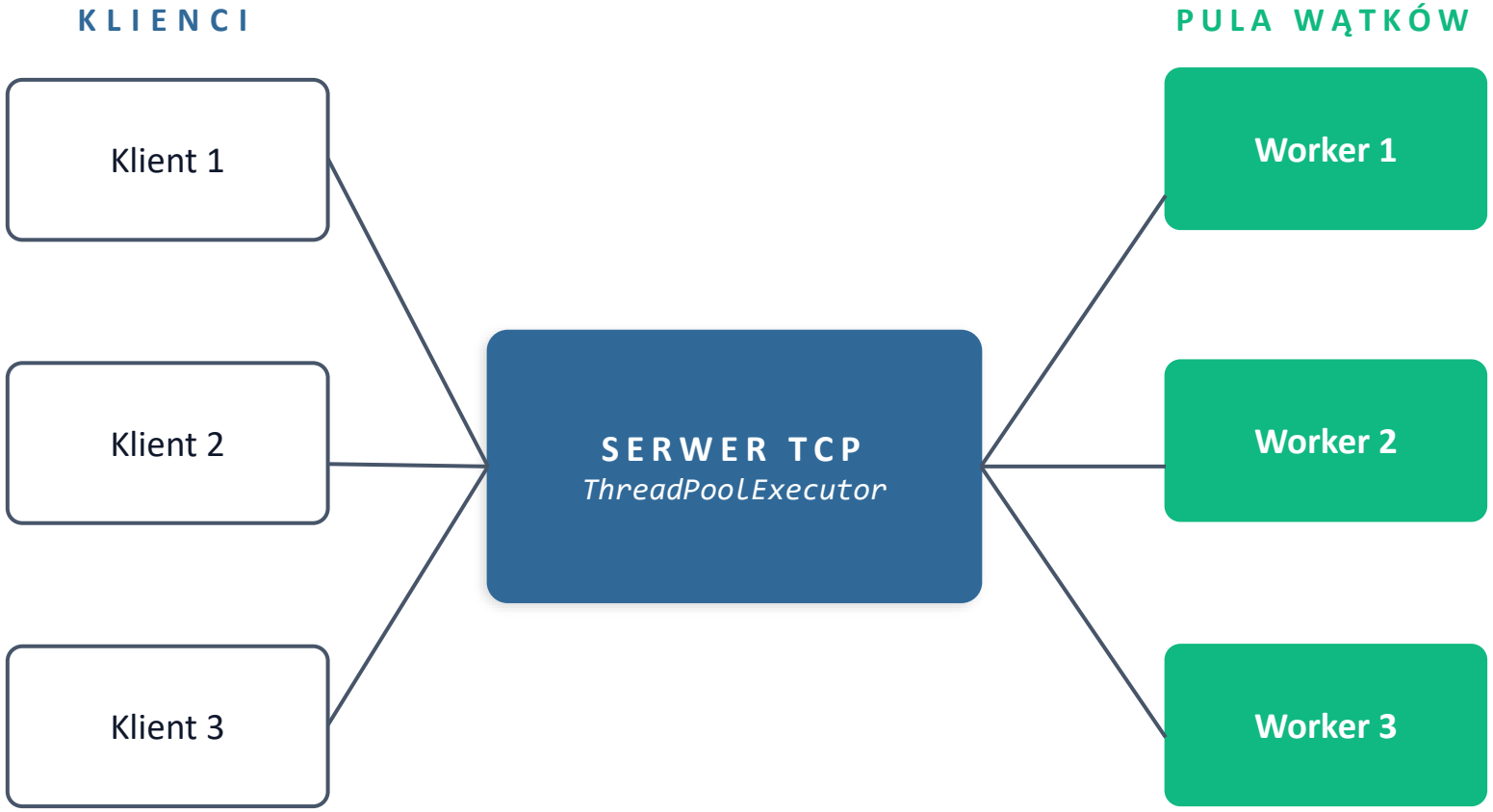
`future.exception()`

pobiera wyjątek (jeśli wystąpił)

`as_completed(futures)`

iteruje gdy któryś skończy

Case study: serwer TCP w telekomunikacji



Scenariusz

Serwer TCP odbiera połączenia od wielu klientów jednocześnie.

Dla każdego połączenia bierze wolnego workera z puli - obsługa równolegle bez blokowania innych klientów.

Dlaczego wątki?

Komunikacja sieciowa to zadanie I/O-bound - GIL jest zwalniany podczas `recv()`.

Serwer TCP - implementacja z pulą wątków

```python
import socket
from concurrent.futures import ThreadPoolExecutor

def obsluz_klienta(client_socket, address):
    print(f"Klient {address} podłączony")
    while True:
        dane = client_socket.recv(1024)   # I/O - GIL zwalniany
        if not dane:
            break
        client_socket.send(dane.upper())   # echo wielkimi
    client_socket.close()

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('localhost', 8080))
server.listen()

with ThreadPoolExecutor(max_workers=10) as pool:
    while True:
        client_socket, address = server.accept()
        pool.submit(obsluz_klienta, client_socket, address)
```

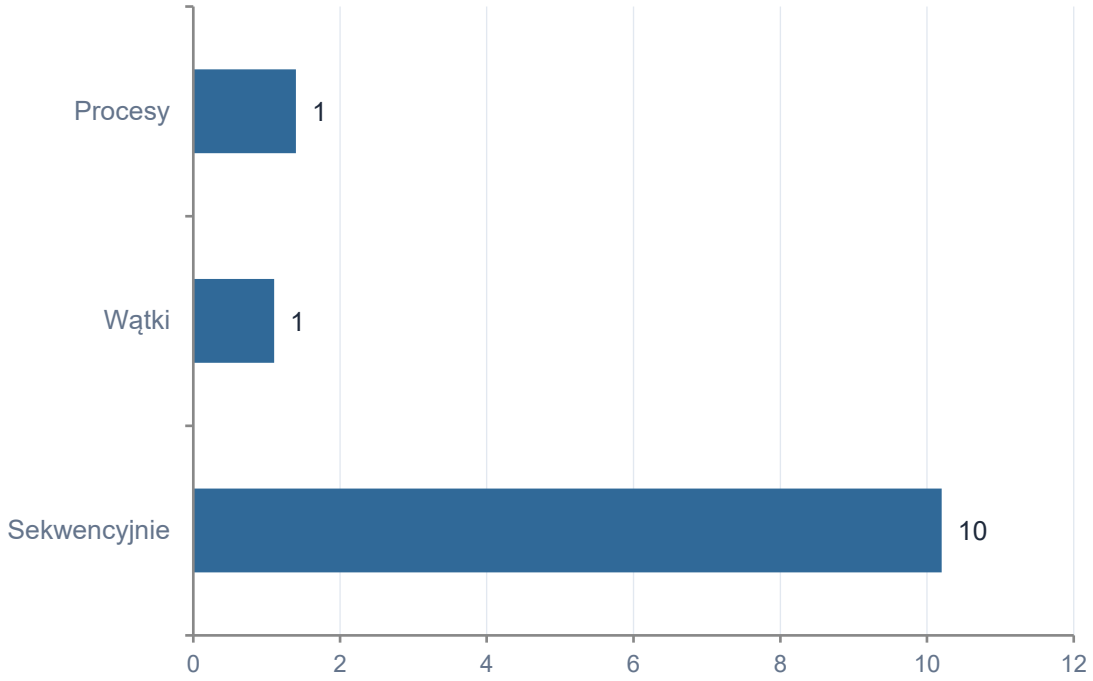
Anatomia kodu

1. **socket.accept() w pętli**
   główny wątek czeka na klientów
2. **executor.submit() dla klienta**
   obsługa trafia do wolnego workera
3. **recv() / send() = I/O**
   GIL zwalniany - inne wątki pracują
4. **max_workers = limit klientów**
   chroni serwer przed przeciążeniem

Benchmark - wątki vs procesy vs sekwencyjnie

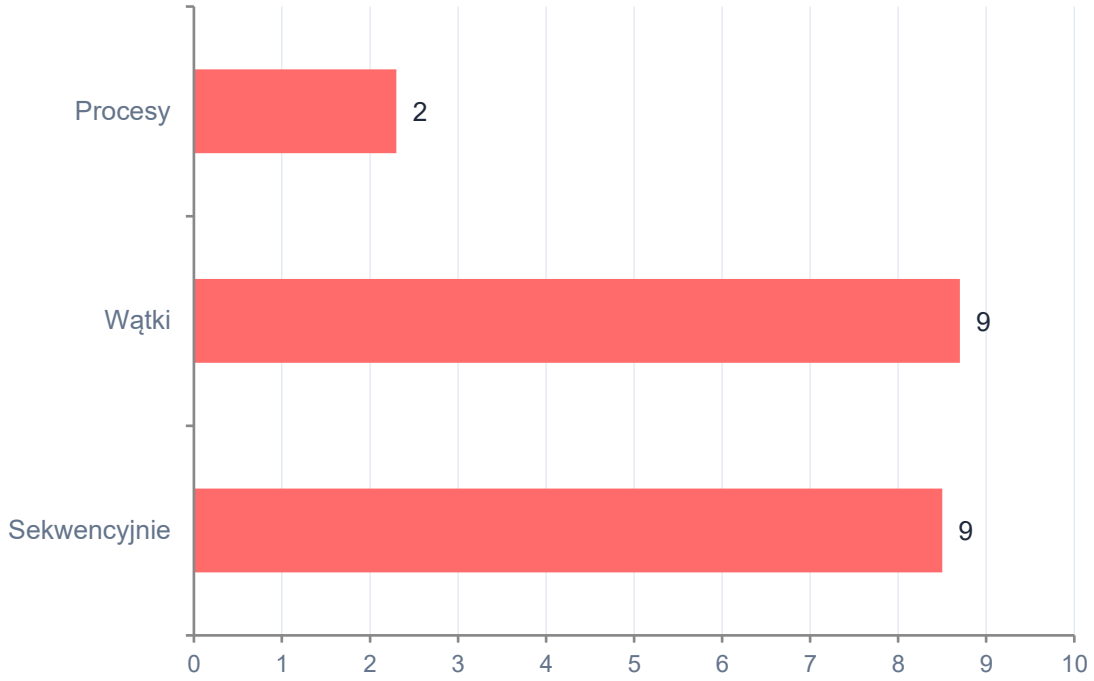
I/O-bound

10 × HTTP request (sleep 1s każdy)



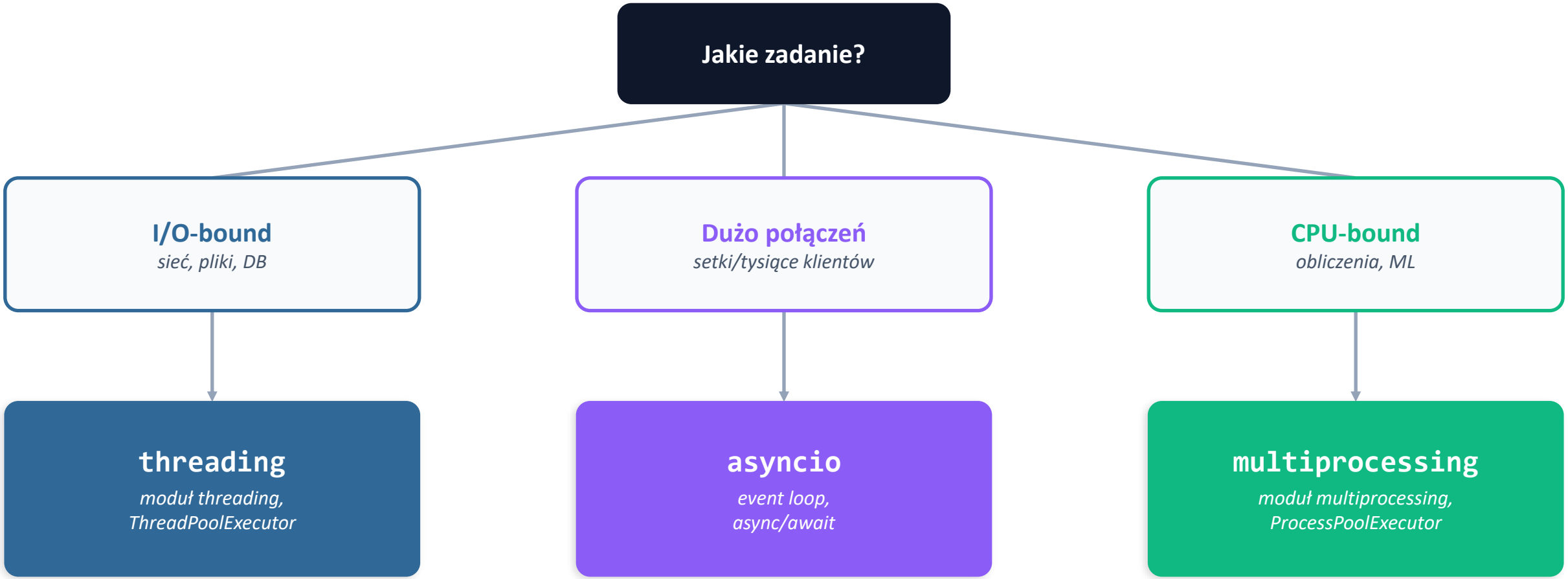
CPU-bound

10 × suma 10 mln liczb



Morał: wątki przyspieszają I/O (~10×), ale w obliczeniach CPU-bound GIL je blokuje - dopiero procesy dają przyspieszenie.

Kiedy czego używać



asyncio - alternatywa dla wielu połączeń

```python
import asyncio

async def pobierz(url):
    print(f"Start: {url}")
    await asyncio.sleep(1)    # nieblokujące czekanie
    print(f"Koniec: {url}")
    return f"dane z {url}"

async def main():
    # 3 zadania współbieżnie w JEDNYM wątku
    wyniki = await asyncio.gather(
        pobierz("a.com"),
        pobierz("b.com"),
        pobierz("c.com"),
    )
    print(wyniki)

asyncio.run(main())   # uruchamia event loop
```

Cechy asyncio

`async def`

definiuje korutynę (lekkie zadanie)

`await`

oddaje sterowanie - daj czas innym

`asyncio.gather()`

uruchamia wiele zadań współbieżnie

`asyncio.run()`

uruchamia event loop z funkcją main

`1 wątek = tysiące zadań`

idealne dla setek/tysięcy klientów

Antypatterns - czego unikać

- ✗ **Globalna zmienna bez Locka**
  gwarantowany race condition - wcześniej czy później
- ✗ **Lock trzymany zbyt długo**
  wąskie gardło - inne wątki tracą sens istnienia
- ✗ **Zagnieżdżone locki w różnej kolejności**
  prosta droga do deadlocka
- ✗ **Wyjątek w wątku zjadany po cichu**
  bez logowania błąd znika - debugowanie piekłem
- ✗ **daemon=True dla ważnych operacji**
  wątek zostanie zabity w środku pracy przy zamknięciu

Antypatterns z kodem - błąd vs naprawa

Antypattern	Błąd	Naprawa

1. Brak Locka
race condition
	`counter = 0`
	`def increment():`
	`  global counter`
	`  counter += 1  # race!`
		`lock = Lock()`
		`def increment():`
		`  global counter`
		`  with lock:`
		`  counter += 1`

2. Lock za długo
I/O w sekcji krytycznej
	`with lock:`
	`  fetch_url()  # 5s!`
	`  counter += 1`
		`data = fetch_url()  # poza`
		`with lock:`
		`  counter += 1`

3. Locki w różnej kolejności
deadlock
	`def w1():`
	`  with a: with b: ...`
	`def w2():`
	`  with b: with a: ...`
		`# zawsze a → b`
		`def w1():`
		`  with a: with b: ...`
		`def w2():`
		`  with a: with b: ...`

4. Wyjątek zjadany
silent error
	`try:`
	`  operacja()`
	`except: pass  # znika!`
		`try:`
		`  operacja()`
		`except Exception:`
		`  logging.exception(...)`

5. daemon=True
dla ważnych operacji
	`t = Thread(`
	`  target=zapis_pliku,`
	`  daemon=True)  # zabity!`
		`t = Thread(`
		`  target=zapis_pliku)`
		`t.start()`
		`t.join()  # czeka`

Podsumowanie i dobre praktyki

1. **Używaj kontekstu with lock:**
   zapewnia zwolnienie blokady także przy wyjątku
2. **Preferuj Queue nad zmienne wspólne**
   kolejka jest wątkowo bezpieczna, mniej błędów
3. **Stała kolejność akwizycji blokad**
   globalny porządek eliminuje deadlocki
4. **ThreadPoolExecutor nad ręczne wątki**
   mniej boilerplate'u, łatwiejsze zarządzanie
5. **Wątki do I/O, procesy do CPU**
   GIL blokuje równoległość wątków w obliczeniach

Dziękujemy za uwagę

Źródła:

Dokumentacja Python - threading, queue, concurrent.futures
docs.python.org/3/library/threading.html
Real Python - An Intro to Threading in Python
realpython.com/intro-to-python-threading